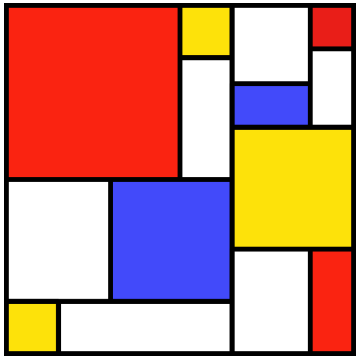
SLiM

Workshop Series



The Population Hierarchy

Benjamin C. Haller

Messer Lab, Cornell University

Individuals

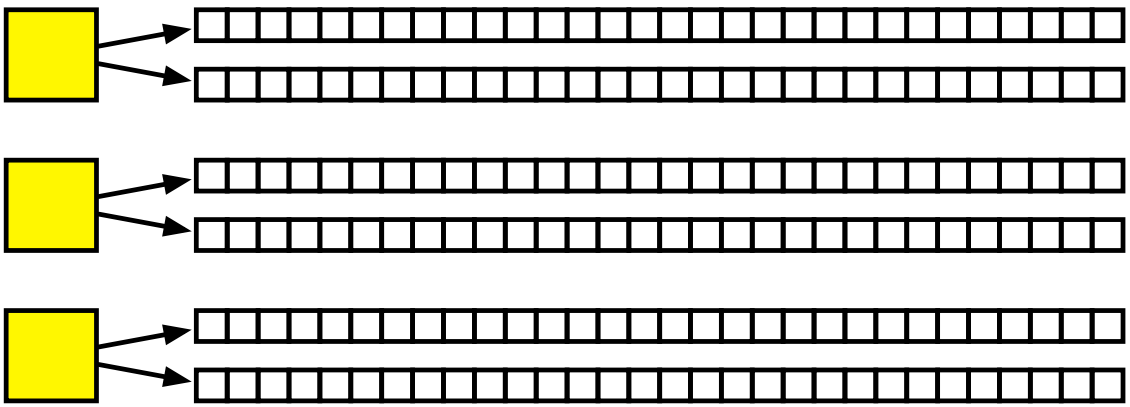
- **Individuals** (class `Individual`) are organisms
- Individuals are born, reproduce, die, …
- Each individual has two **haplosomes**

Individuals

- **Individuals** (class `Individual`) are organisms
- Individuals are born, reproduce, die, …
- Each individual has two **haplosomes**
- …wait, what's a haplosome??

Haplosomes

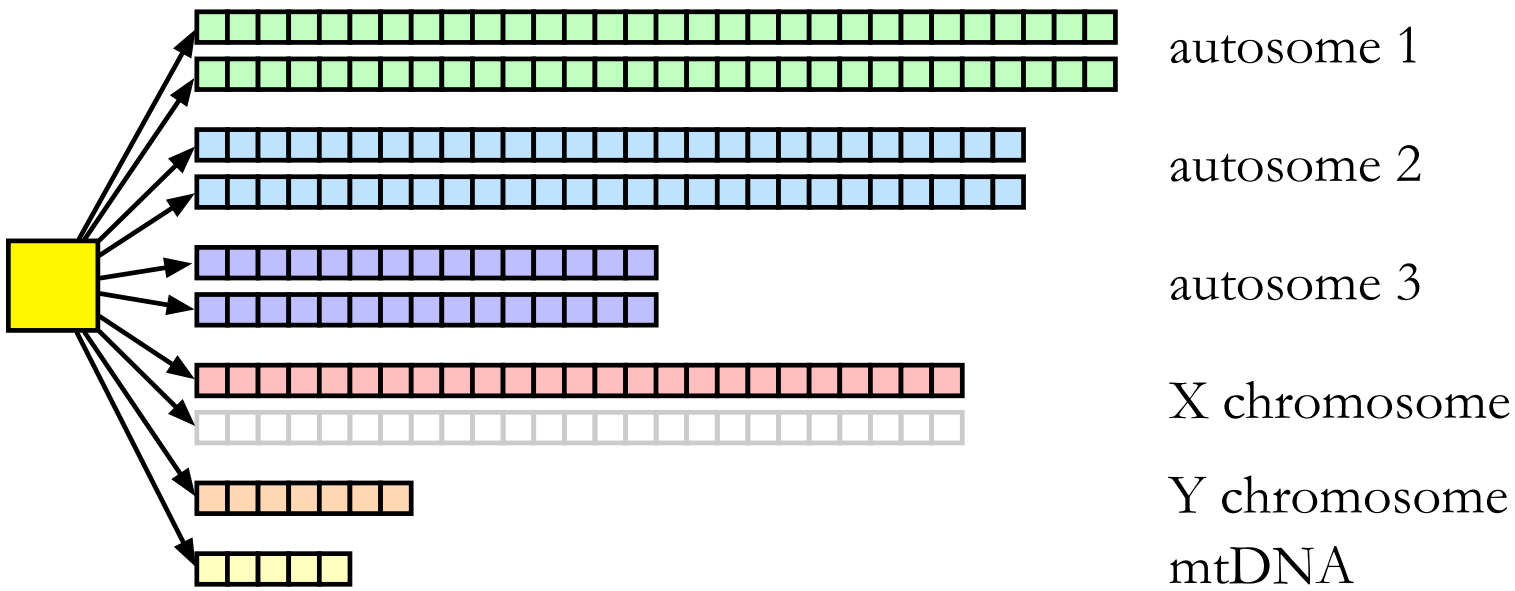
- Before SLiM 5, there was only one chromosome
- Individuals thus looked like this:



- Each diploid individual had two genetic “things”
- Those “things” were called “genomes” in SLiM

Haplosomes

- But "genome" assumes a single chromosome
- In SLiM 5, you can have more than one:



- What should we call all these "things"?

Haplosomes

- What to call all these "things" containing the genetics of a multi-chromosome individual?
- There's no simple term in biology for them!
- "strand", "chromatid", "chromosome" ⇒ nope
- In practice, people use language like:

  "one of the two homologous copies of chromosome 4"

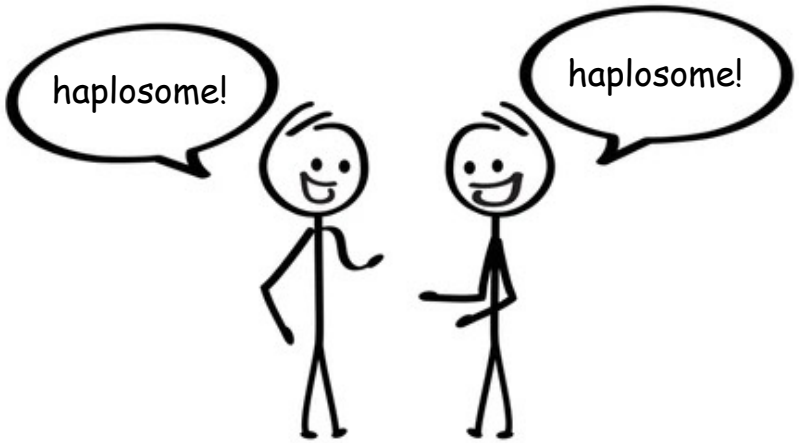
  "the maternal copy of the X chromosome"
- A new term for this concept is needed!

Haplosomes

- The new term: **haplosome**
- A haplosome is: *one of the homologous versions of a given chromosome in a given individual*
- Greek roots: *ἁπλόος* "single" + *σῶμα* "body"
- The term "haplosome" is now central to SLiM

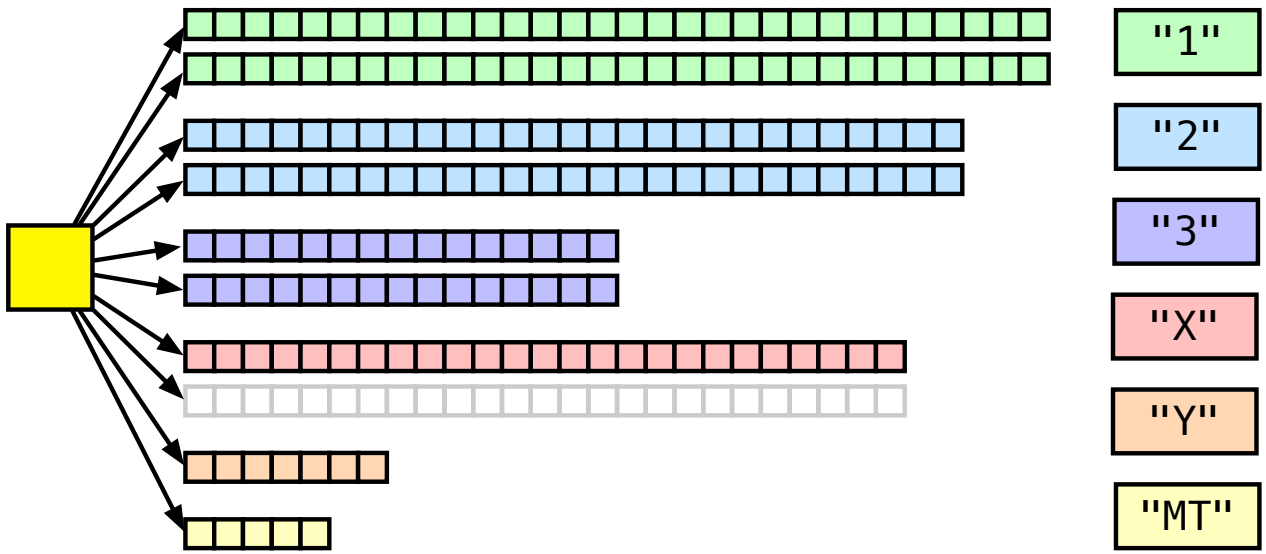
Haplosomes

- The new term: **haplosome**
- A haplosome is: *one of the homologous versions of a given chromosome in a given individual*
- Greek roots: *ἁπλόος* "single" + *σῶμα* "body"
- The term "haplosome" is now central to SLiM



Multiple Chromosomes

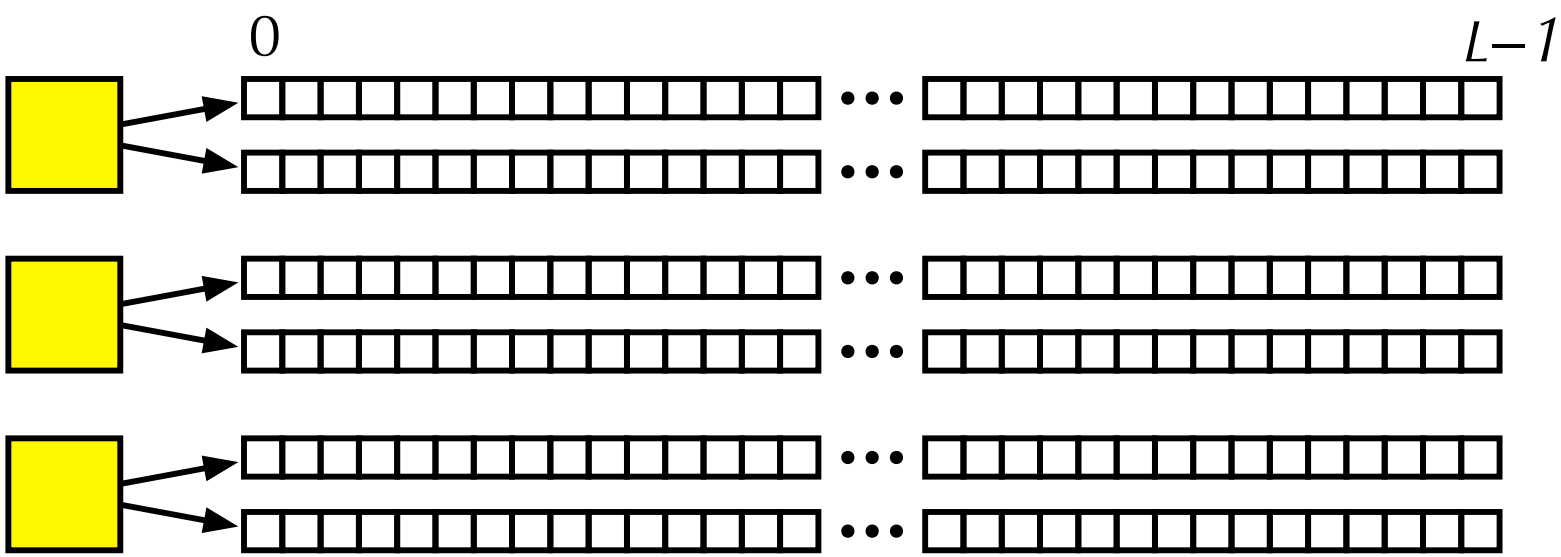
- So there can be multiple chromosome objects:



- Each haplosome has an associated chromosome
- Each chromosome has a length, type, ploidy…
- But let's go back to the simple picture for now!

Individuals

- **Individuals** (class `Individual`) are organisms
- Individuals are born, reproduce, die, …
- Each individual has two **haplosomes**
- Each haplosome has L discrete base positions

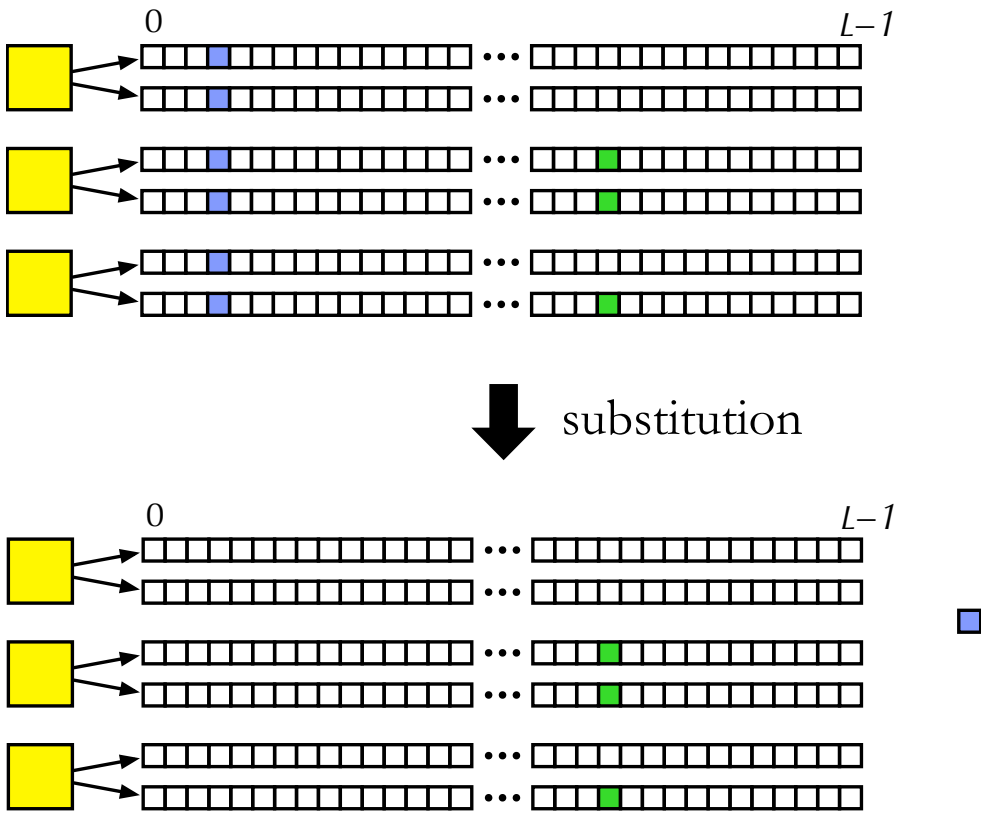


Mutations

- **Mutations** (class `Mutation`) live in haplosomes
- Haplosomes begin empty (the *ancestral* state)
- Mutations represent a *non-ancestral* allele (SNP)
- Mutations have properties: selection coefficient

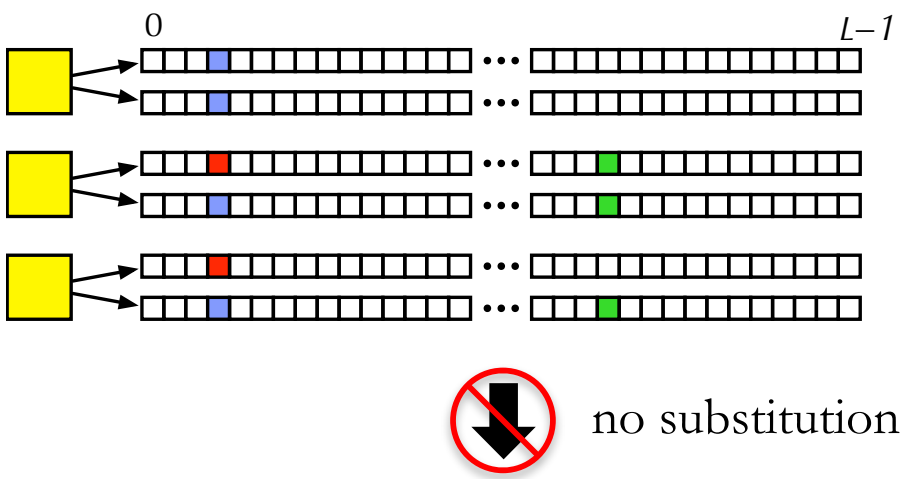
Substitutions

- **Substitutions** (class `Substitution`) are fixed
- Fixed mutations become substitutions by default



Substitutions

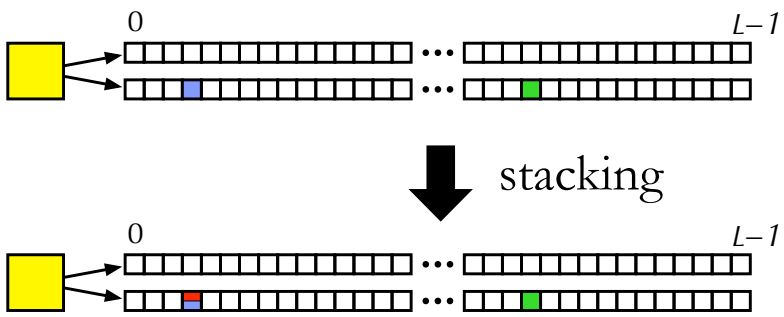
- Every **mutational lineage** is distinct!
- Even with identical selection coefficients, etc.



- This models "identity by descent" (IBD)
  - "unique down" mutations for identity by state (IBS)

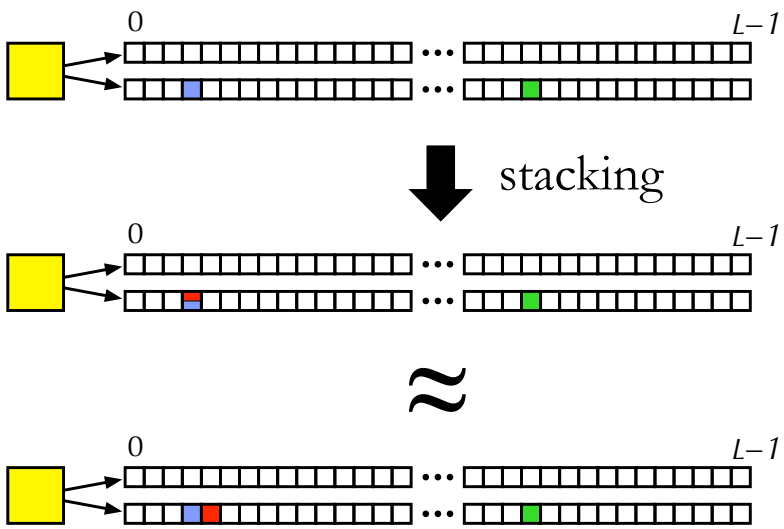
Mutation stacking

- If mutation A is in a haplosome at position *P*…
- And then mutation B occurs at *P*…
- By default, *both* are kept; they **stack**
- Stacking policy can be modified, however



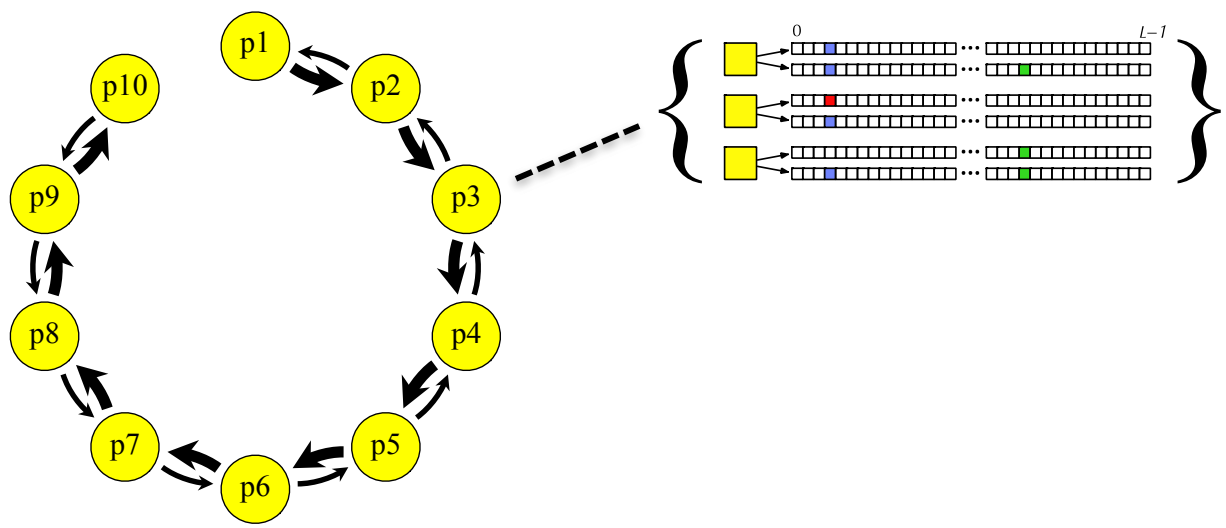
Mutation stacking

- The default behavior of stacking:
  - matches "infinite-sites" analytical models
  - approximates adjacent linked mutations
  - generally makes little difference (but can be changed)



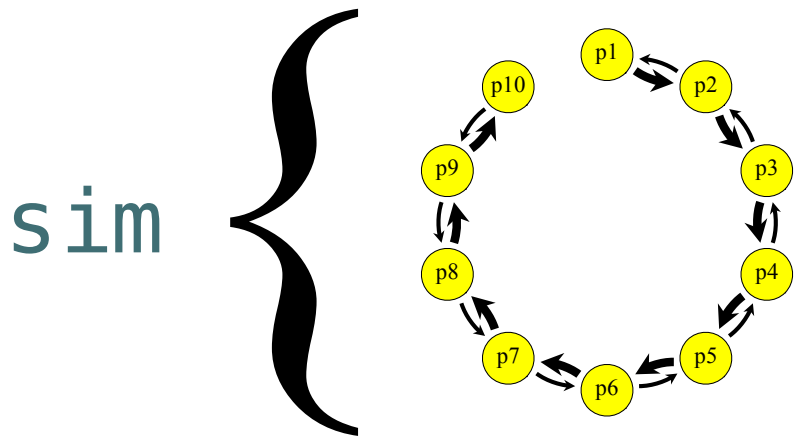
Subpopulations and migration

- **Subpopulations** (`Subpopulation`) contain individuals
- Any number of subpopulations can exist
- They can be connected by **migration** in any way
- Population structure can change over time



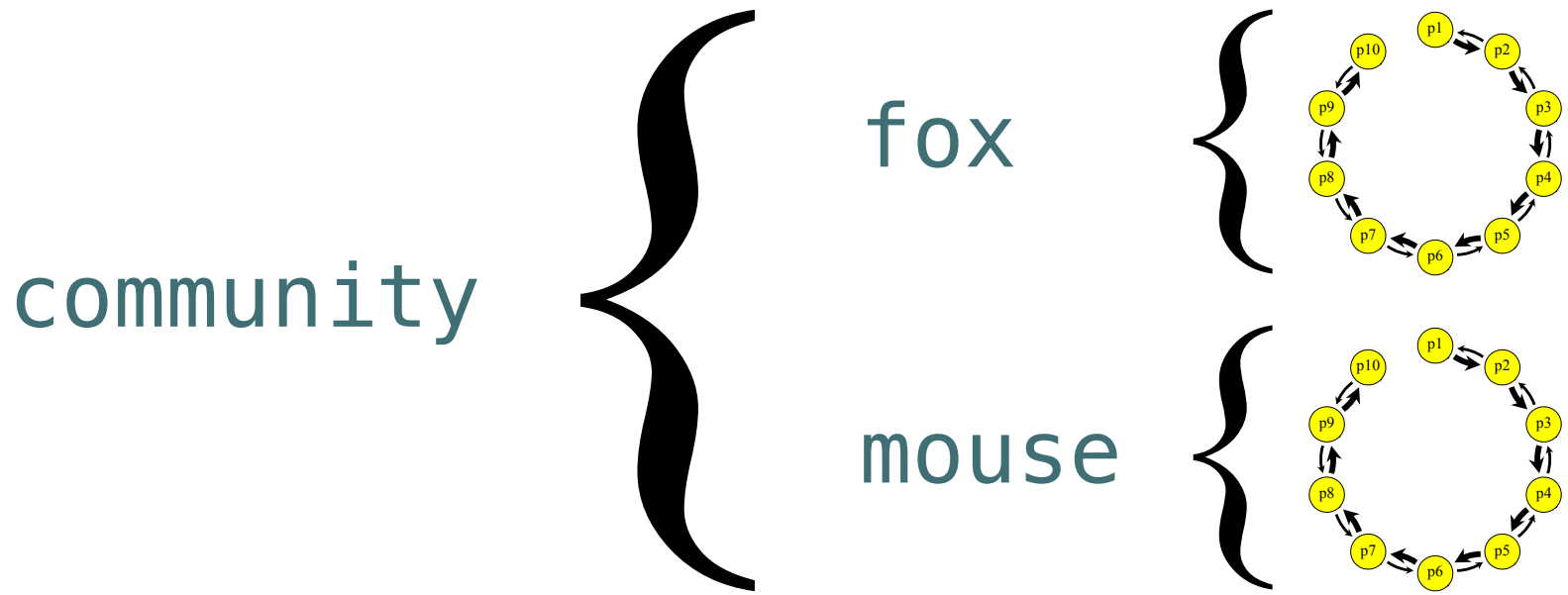
The population and species

- The **population** contains subpopulations
- Only one population can exist (per species)
- It is managed by its **species** (`Species`)
- The default species is named `sim`



The community

- There can be more than one species in a model
- Each species has its own population
- Species belong to the **ecological community**, named `community` (`Community`)



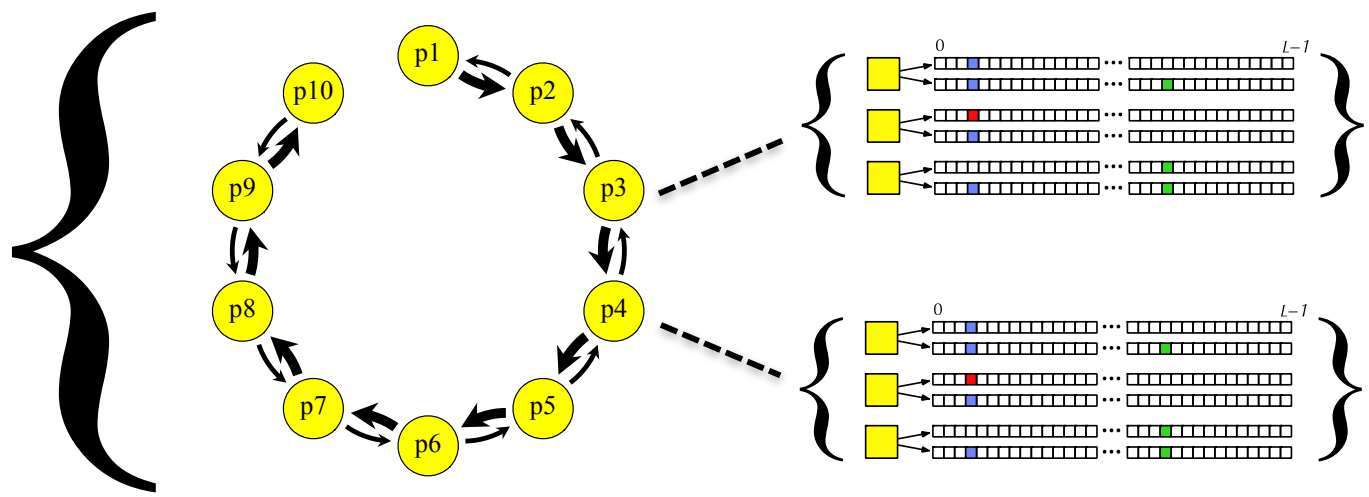
The community hierarchy

- The **community** contains one or more **species**
- Each species contains **subpopulations**
- Each subpopulation contains **individuals**
- Each individual contains **haplosomes**
- Each haplosome contains **mutations**

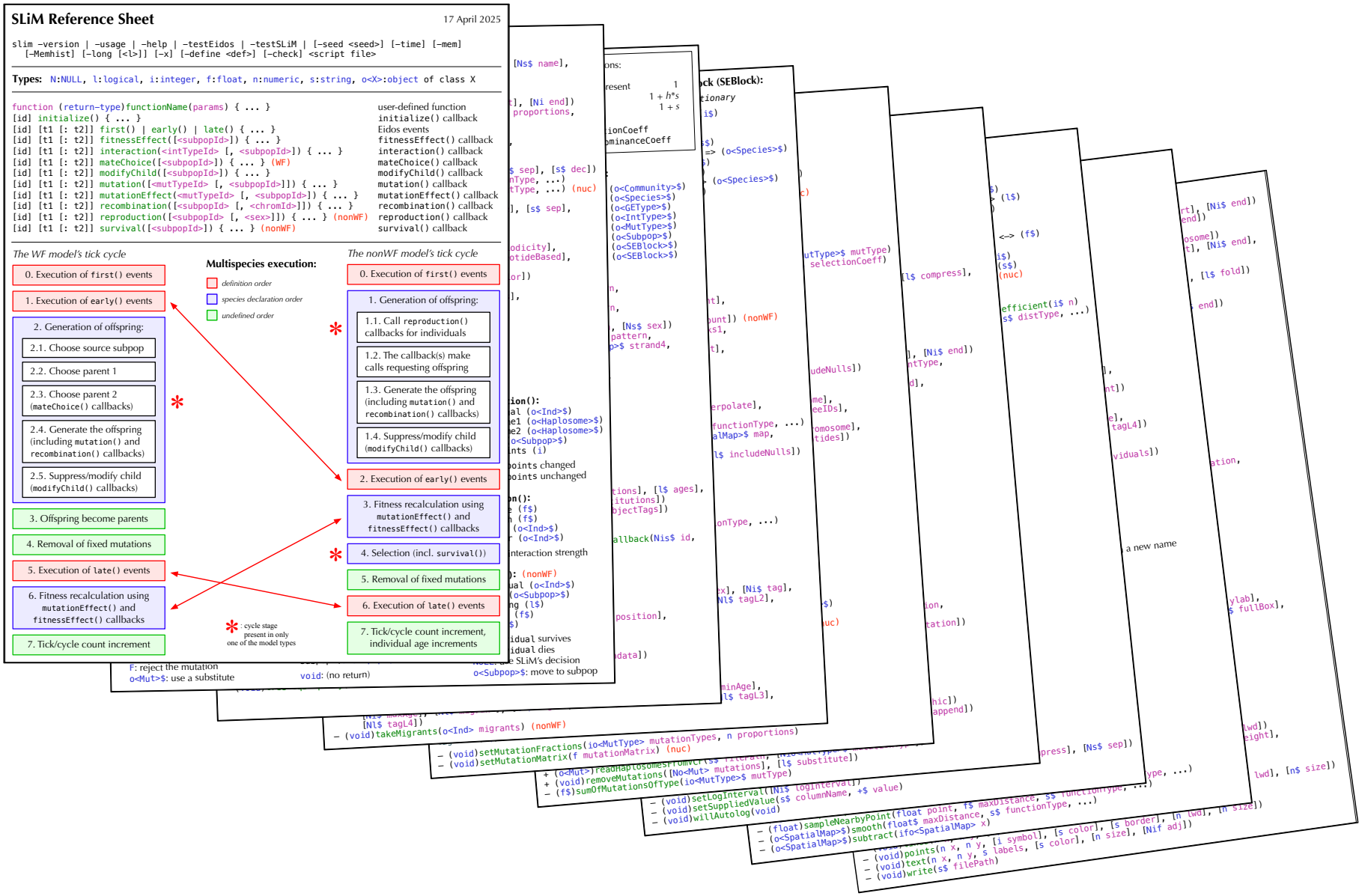
carrot

rabbit

coyote



SLiM reference sheet



SLiM Reference Sheet

17 April 2025

```
slim -version | -usage | -help | -testEidos | -testSLiM | [-seed <seed>] [-time] [-mem]
  [-Memhist] [-long [<l>]] [-x] [-define <def>] [-check] <script file>
```

Types: `N:NULL`, `l:logical`, `i:integer`, `f:float`, `n:numeric`, `s:string`, `o<X>:object` of class X

`function (return-type)functionName(params) { ... }`	user-defined function
`[id] initialize() { ... }`	`initialize()` callback
`[id] [t1 [: t2]] first() | early() | late() { ... }`	Eidos events
`[id] [t1 [: t2]] fitnessEffect([<subpopId>]) { ... }`	`fitnessEffect()` callback
`[id] [t1 [: t2]] interaction(<intTypeId> [, <subpopId>]) { ... }`	`interaction()` callback
`[id] [t1 [: t2]] mateChoice([<subpopId>]) { ... }` (WF)	`mateChoice()` callback
`[id] [t1 [: t2]] modifyChild([<subpopId>]) { ... }`	`modifyChild()` callback
`[id] [t1 [: t2]] mutation([<mutTypeId> [, <subpopId>]]) { ... }`	`mutation()` callback
`[id] [t1 [: t2]] mutationEffect(<mutTypeId> [, <subpopId>]) { ... }`	`mutationEffect()` callback
`[id] [t1 [: t2]] recombination([<subpopId> [, <chromId>]]) { ... }`	`recombination()` callback
`[id] [t1 [: t2]] reproduction([<subpopId> [, <sex>]]) { ... }` (nonWF)	`reproduction()` callback
`[id] [t1 [: t2]] survival([<subpopId>]) { ... }` (nonWF)	`survival()` callback

The WF model's tick cycle

- 0. Execution of `first()` events
- 1. Execution of `early()` events
- 2. Generation of offspring:
  - 2.1. Choose source subpop
  - 2.2. Choose parent 1
  - 2.3. Choose parent 2 (`mateChoice()` callbacks) *
  - 2.4. Generate the offspring (including `mutation()` and `recombination()` callbacks)
  - 2.5. Suppress/modify child (`modifyChild()` callbacks)
- 3. Offspring become parents
- 4. Removal of fixed mutations
- 5. Execution of `late()` events
- 6. Fitness recalculation using `mutationEffect()` and `fitnessEffect()` callbacks
- 7. Tick/cycle count increment

Multispecies execution:

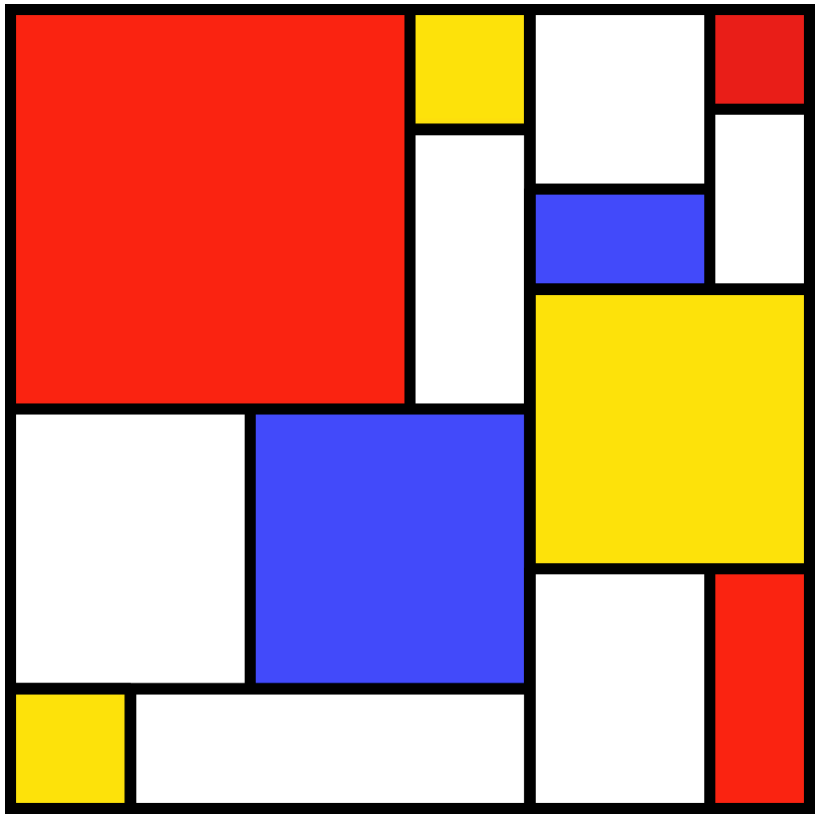
- *definition order*
- *species declaration order*
- *undefined order*

* : cycle stage present in only one of the model types

The nonWF model's tick cycle

- 0. Execution of `first()` events
- 1. Generation of offspring: *
  - 1.1. Call `reproduction()` callbacks for individuals
  - 1.2. The callback(s) make calls requesting offspring
  - 1.3. Generate the offspring (including `mutation()` and `recombination()` callbacks)
  - 1.4. Suppress/modify child (`modifyChild()` callbacks)
- 2. Execution of `early()` events
- 3. Fitness recalculation using `mutationEffect()` and `fitnessEffect()` callbacks
- 4. Selection (incl. `survival()`) *
- 5. Removal of fixed mutations
- 6. Execution of `late()` events
- 7. Tick/cycle count increment, individual age increments

`F`: reject the mutation
`o<Mut>$`: use a substitute
`void`: (no return)
`o<Subpop>$`: move to subpop



Copyright © 2020–2025 Benjamin C. Haller. All Rights Reserved.